EXPERIMENT 15

Report and presentation on career pathways and the future of low-code development

241401117

Submitted by	**Tharika R**
Register Number	**241401117**
Department	**Computer Science and Business Systems**

1. Aim

The aim of this experiment is to study the career opportunities available to low-code developers and to examine the likely direction of low-code and no-code platforms across the software industry. The study treats low-code as a broad field made up of many platforms, rather than as the property of one product, and concludes with a written report and a ten-slide presentation outline.

The experiment also explains how low-code development, pro-code extensions, process automation and AI-assisted application building are becoming connected parts of a single development landscape. Visual tools now produce much of the routine structure of an application, scripting and APIs extend what those tools can do, automation links applications into complete business processes, and AI assistants increasingly draft the first version of the work for people to review.

Finally, the study relates these developments to the practical skills gained through the course experiments, particularly the Bank Loan Application Form, which combined form design, data capture, workflow logic and a pro-code extension. The findings help in deciding which skills to build next and which roles to aim for as the boundaries between low-code, pro-code and AI-assisted development narrow.

2. Procedure

1. **Study the evolution of development.** Application development was traced from traditional hand coding, through rapid-development tools and visual low-code platforms, to AI-assisted building (Section 3.1).
2. **Compare the platforms.** Seven widely used low-code platforms were compared by general focus and typical use to show that different products serve different application areas (Section 3.2).
3. **Identify industry trends.** Current trends were listed together with their possible effects on software careers (Section 3.3).
4. **Map career opportunities.** Roles were arranged into entry-level, intermediate and senior levels, including possible moves into other careers (Section 3.4).
5. **Compare professional roles.** The responsibilities and key skills of six representative roles were tabulated side by side (Section 3.5).
6. **Develop the skill stack and roadmap.** The required skills were arranged in layers and converted into a five-stage learning plan (Sections 3.6 and 3.7).
7. **Examine the risks.** Shadow IT, security, AI-generated output quality, vendor lock-in and a changing job market were analysed with practical mitigation measures (Section 3.8).
8. **Design a future workflow.** A possible AI-assisted development cycle with human review and governed release was drawn (Section 3.9).
9. **Prepare the presentation.** A ten-slide outline was written so that the findings can be presented clearly in class (Section 3.10).

Note: *This study draws on general knowledge of the industry. Trends, forecasts and any figures change quickly and must be checked against current, original sources before being cited as facts. The timeline, roles and roadmap are general guides and do not predict the future of any particular company or individual.*

241401117

3. Output

3.1 Evolution of Application Development

Application development has moved from being the exclusive work of specialist programmers to a shared activity in which visual tools, business users and AI assistants all take part. Each stage in this change reduced the effort needed to produce a working application, but each also altered who is able to build software and how its quality is checked. Figure 3.1 summarises five broad stages; the stages overlap in practice, and organisations often use several of them at once.

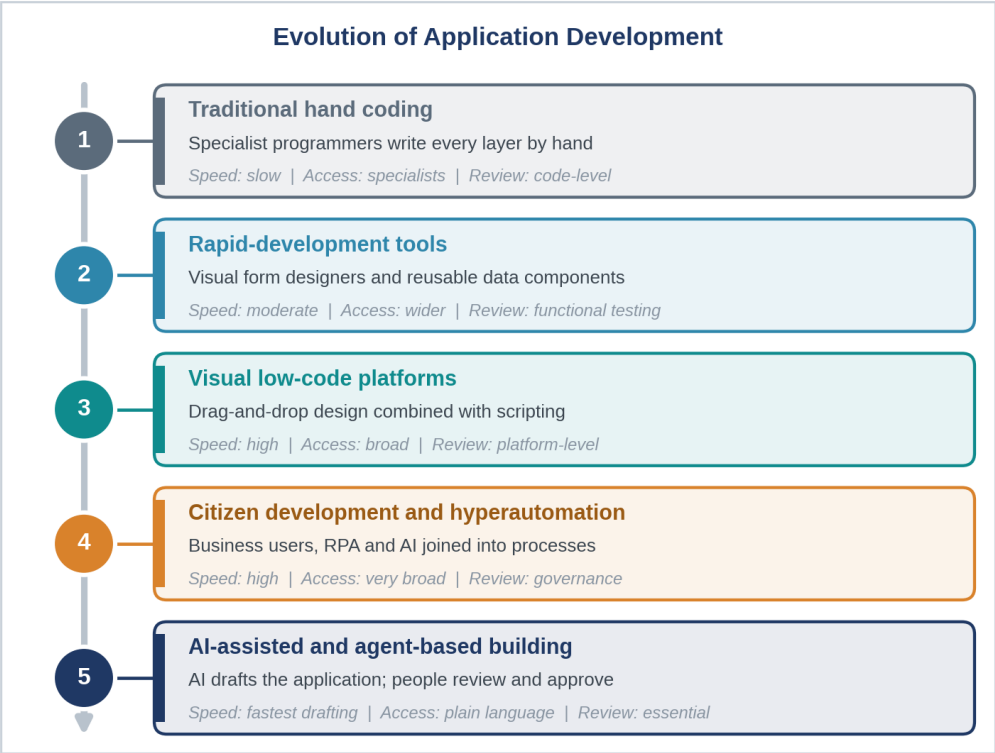


Figure 3.1: Five stages in the evolution of application development

In **traditional hand coding**, speed is limited because every screen, database call and rule is written manually, and only trained programmers can contribute; review takes place at code level. **Rapid-development tools** introduced visual form designers and reusable components, which shortened delivery and widened participation slightly, while functional testing became the main safeguard. **Visual low-code platforms** combine drag-and-drop design with scripting, so delivery is much faster and analysts can contribute, but the platform's behaviour must still be understood and reviewed. **Citizen development and hyperautomation** extend building to business users and link applications with RPA and AI, which makes governance essential. In **AI-assisted and agent-based building**, a draft can be produced from a plain-language description, giving the fastest start and the widest access, yet careful human review of logic, data handling and security becomes more important than ever.

3.2 The Platform Landscape Beyond Zoho Creator

Low-code is not a single product but a family of platforms with different strengths. Some are designed around business forms and reports, some around large enterprise systems, others around process automation, customer data or spreadsheets. Table 3.2 compares seven well-known examples, so that a learner can see where each is normally applied and why experience with one platform, such as Zoho Creator, is only the starting point.

Platform	General Focus	Typical Use
Zoho Creator	Business application builder that works closely with the wider Zoho product suite.	Forms, workflows and reports for small and mid-sized organisations.
Microsoft Power Apps	Apps and automations connected to Microsoft 365, Dataverse and Azure services.	Internal tools and approval flows in organisations that already use Microsoft products.
OutSystems	Enterprise-grade platform with strong support for extension through custom code.	Large, scalable and complex enterprise applications.
Mendix	Model-driven development environment suited to collaboration between business and IT teams.	Enterprise and industrial applications built by mixed teams.
Appian	Process-automation and case-management platform.	Regulated, workflow-heavy operations such as claims handling and compliance.
Salesforce Platform	Custom applications built on top of customer relationship data.	Customer-facing, sales and service applications.
Google AppSheet	Data-driven apps generated from spreadsheets and cloud data sources.	Simple team tools and mobile field-data collection.

Table 3.2: Comparison of selected low-code platforms

The table distinguishes business applications (Zoho Creator), enterprise development (OutSystems and Mendix), process automation (Appian), customer-data applications (Salesforce Platform) and spreadsheet-based tools (Google AppSheet). Despite these differences, many skills transfer from one product to another: data modelling, form and workflow design, role-based access, API integration and testing habits are needed everywhere. A developer who understands these ideas can usually learn a new platform's interface and terminology far faster than a beginner, so depth in fundamentals is more valuable than loyalty to one vendor.

3.3 Trends Shaping the Future

The low-code industry is changing as AI features, integration standards and governance practices mature. Several trends are frequently discussed, and Table 3.3 describes each in terms of its meaning and its possible effect on careers. These points are general observations drawn from common industry discussion; they are not verified forecasts and no statistics are claimed.

Trend	What It Means	Effect on Careers
AI-assisted building	Applications, data structures and logic can be drafted from a plain-language description.	Less routine construction; more reviewing, design and testing of generated work.
AI agents inside applications	Agents carry out actions within workflows instead of only offering suggestions.	New work in designing, restricting and monitoring agent behaviour.
Governed citizen development	Business users build applications within rules and guardrails set by IT.	Greater need for platform owners, policy writers and centres of excellence.
Hyperautomation	Low-code, RPA and AI are combined to automate complete processes.	Automation engineers and process designers become more valuable.
Legacy modernisation	Older systems are wrapped, extended or replaced with low-code front ends.	Integration and migration skills gain importance.
API-first and composable applications	Applications are assembled from reusable services reached through APIs.	API design and integration skills transfer across platforms.
Fusion teams	Business and technical staff build solutions together as one team.	Communication and requirement-gathering matter as much as tools.

Table 3.3: Industry trends and their possible effect on careers

Taken together, these trends shift development work from typing every element towards specifying, checking and connecting. AI-assisted building and agents place more weight on testing and on defining what an application may and may not do. Hyperautomation, legacy modernisation and API-first design increase integration responsibilities, because value lies in linking systems reliably. Governed citizen development and fusion teams raise the importance of governance, communication and shared standards. Because any of these directions may develop differently from expectation, they should be treated as reasonable signals rather than certainties.

3.4 Career Pathways

A low-code career rarely follows one fixed route. Many people begin as citizen developers or business analysts, or as junior developers on a single platform, then broaden into development, integration or automation, and later move towards architecture, platform leadership or AI-focused design. Figure 3.4 shows common movements between these levels, together with moves into neighbouring careers.

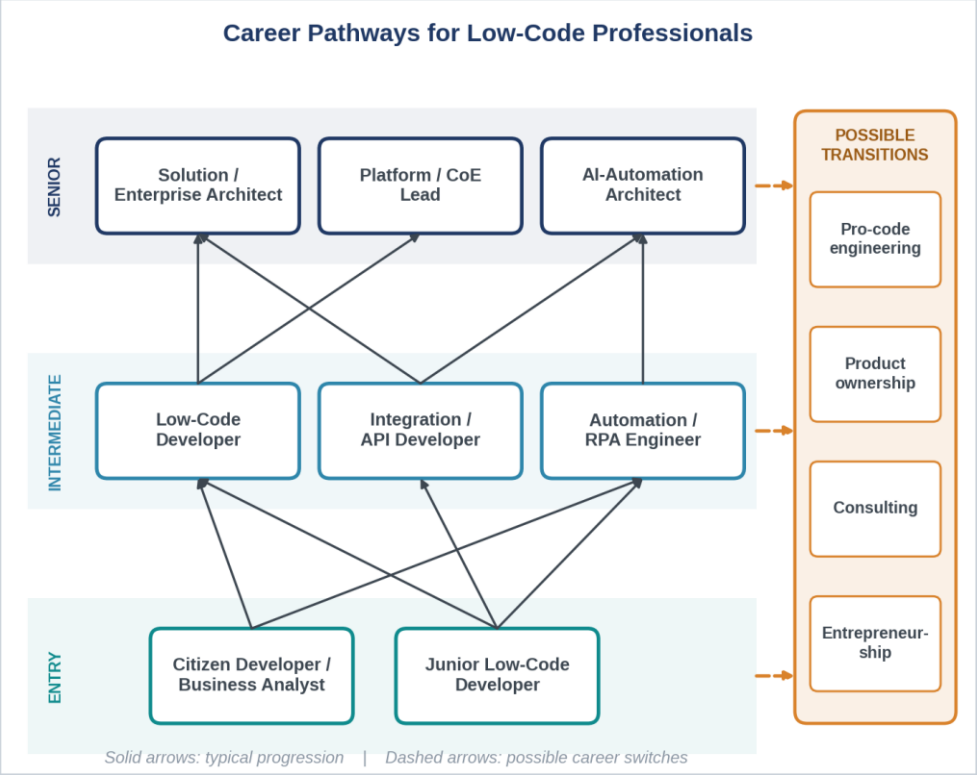


Figure 3.4: Career pathway for low-code professionals

The lower tier contains entry-level roles in which people learn the platform and the business process: the Citizen Developer / Business Analyst and the Junior Low-Code Developer. Experience leads to the intermediate tier, where specialisation begins: the Low-Code Developer builds complete applications, the Integration / API Developer connects them to other systems, and the Automation / RPA Engineer automates end-to-end processes. At the senior tier, the Solution / Enterprise Architect designs overall solutions, the Platform / Centre of Excellence Lead sets standards and supports other builders, and the AI-Automation Architect designs governed solutions that include AI services and agents. The dashed arrows indicate that people may also transfer to pro-code engineering, product ownership, consulting or entrepreneurship, because the problem-solving, integration and requirement-analysis skills they have acquired are widely useful.

3.5 Roles at a Glance

Different low-code roles carry different responsibilities and expect different mixtures of technical, analytical and communication skills. Table 3.5 summarises six representative roles. Job titles vary

between employers, so the table should be read as a guide to typical duties rather than a fixed definition.

Role	Main Responsibilities	Key Skills
Citizen Developer / Analyst	Builds simple applications for their own team and gathers business requirements.	Process thinking, requirement gathering, basic platform use.
Low-Code Developer	Builds forms, workflows, reports and scripted logic for complete applications.	Platform expertise, data modelling, scripting.
Integration / API Developer	Connects applications with other systems through APIs and manages data exchange.	REST, JSON, authentication, error handling.
Automation / RPA Engineer	Automates end-to-end processes, including steps that use AI services.	Workflow design, RPA tools, AI services.
Solution / Enterprise Architect	Designs the overall solution, including its security, structure and governance.	Architecture, security, platform strategy.
Platform / CoE Lead	Sets standards, manages the platform and supports citizen developers.	Governance, training, communication.

Table 3.5: Responsibilities and key skills of low-code roles

Reading down the table, responsibilities widen from building a single application to shaping how many people build and how systems fit together. Technical knowledge deepens in step with this: from basic platform use, to data modelling and scripting, then to APIs and authentication, and finally to architecture and security. Communication grows in importance at both ends, since analysts must understand business needs and leads must explain standards and train others. Governance is minor in the early roles but central in the architect and CoE positions, where decisions affect every application on the platform.

3.6 Skill Stack for a Low-Code Career

A strong low-code career requires more than learning the interface of one platform. Interfaces change, products are replaced and AI tools take over routine steps, whereas an understanding of data, logic, integration and security remains useful throughout. Figure 3.6 arranges the necessary skills in six layers, from basic foundations at the bottom to advanced specialisation at the top.

24140117

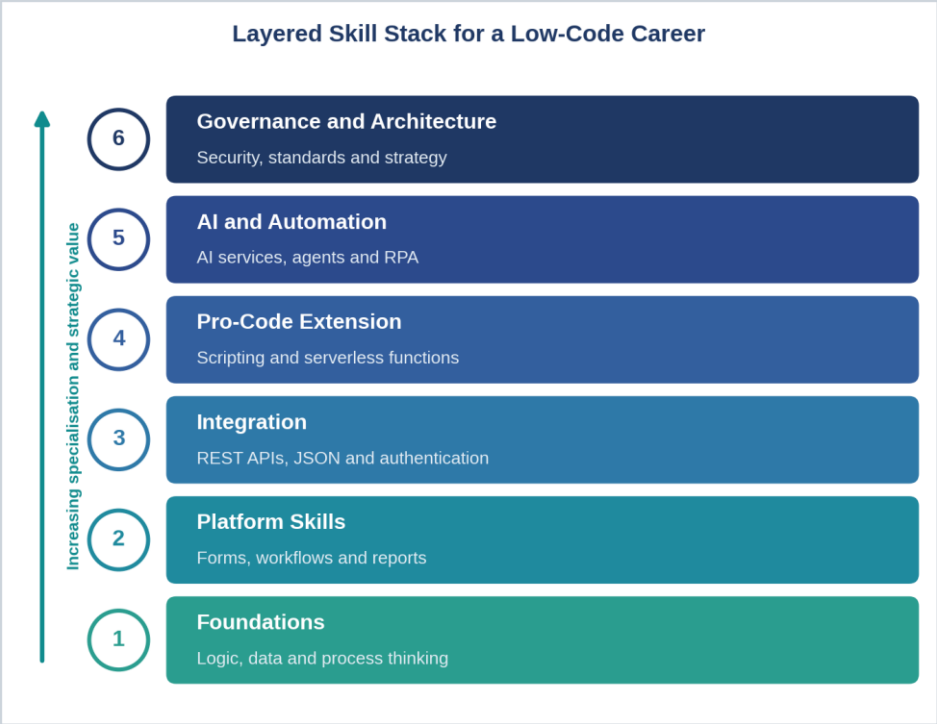


Figure 3.6: Layered skill stack for a low-code career

Foundations (logic, data and process thinking) support the **platform skills** of forms, workflows and reports, because a screen cannot be designed well without understanding the data and process behind it. **Integration** (REST APIs, JSON and authentication) lets those applications exchange information with other systems, and **pro-code extension** (scripting and serverless functions) handles needs that visual tools cannot meet. **AI and automation** build on this base by adding AI services, agents and RPA, and **governance and architecture** sit at the top, because security, standards and strategy can only be judged by someone who understands the layers beneath. As AI tools improve, integration, critical thinking, security and governance remain important: generated output must still be checked for correctness, connected safely to real systems and approved against organisational rules, and this responsibility cannot be delegated to the tool that produced it.

3.7 Learning Roadmap

Low-code skills are best developed progressively, with each stage supported by practical work. Figure 3.7 presents a five-stage roadmap in which students first learn core concepts, then apply them in projects, and later extend them with integration, AI and specialisation.

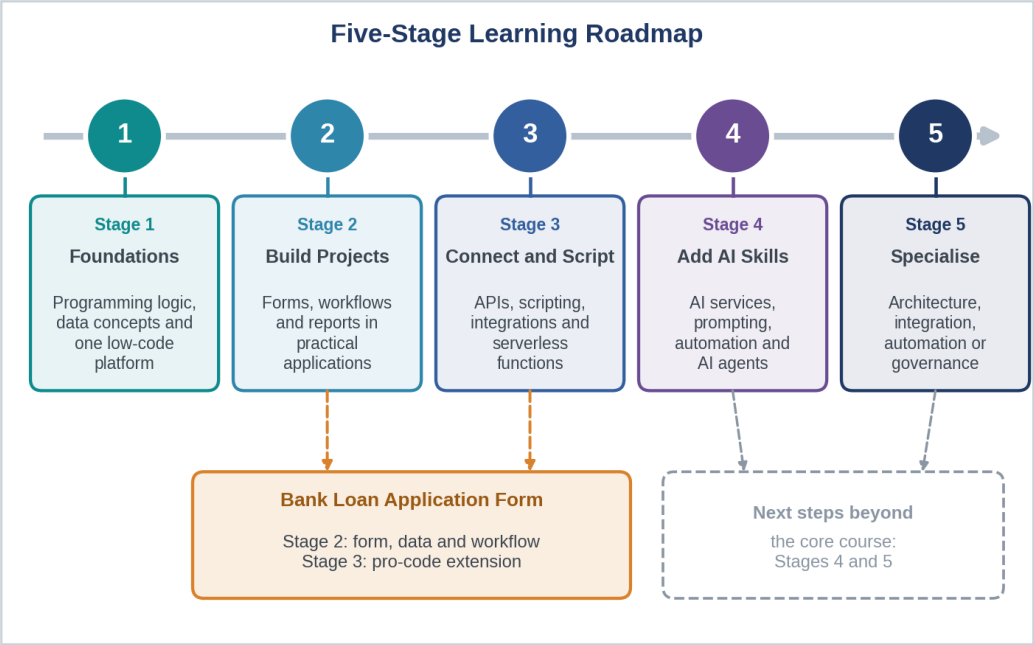


Figure 3.7: Five-stage learning roadmap with course project mapping

Stage 1 builds programming logic, data concepts and familiarity with one low-code platform. Stage 2 applies these through practical projects using forms, workflows and reports, and Stage 3 adds APIs, scripting, integrations and serverless functions. Stage 4 explores AI services, prompting, automation and AI agents, and Stage 5 leads towards specialisation in architecture, integration, automation or platform governance. The Bank Loan Application Form fits mainly into Stages 2 and 3: designing its form and data fields and defining its approval workflow belong to Stage 2, while the pro-code extension that adds custom logic belongs to Stage 3. Extending the same project with an API connection or an AI-assisted checking step would be a natural way to progress into Stages 4 and 5.

3.8 Risks and Challenges

Wider adoption of low-code and AI-assisted development brings real benefits, but it also introduces risks that organisations and individual developers must manage. Table 3.8 describes five important challenges, explains why each matters and suggests practical ways of handling it.

Challenge	Why It Matters	How to Handle It
Shadow IT	Applications built outside IT oversight may be unknown, insecure or duplicated.	Maintain a central application register, use approval procedures and establish a centre of excellence.
Security and compliance	Easy building can hide weak access control and accidental exposure of data.	Apply role-based access, periodic access reviews, security testing and audit trails.
Quality of AI-generated output	Generated logic can appear correct while hiding errors or unsafe assumptions.	Require human review, written test cases and version control for all generated work.
Vendor lock-in	Applications may depend heavily on one platform's tools, licensing and pricing.	Keep core logic behind APIs, keep data exportable and develop transferable skills.
Changing job market	Routine build tasks may shrink as AI tools improve, and job requirements may shift.	Move towards design, integration, governance and AI oversight; keep learning.

Table 3.8: Risks of wider low-code adoption and suggested responses

The mitigation measures in the table share a common principle: ease of building must be matched by discipline in checking, approving and maintaining what is built. Responsible development means testing before release, controlling access to data and documenting decisions so that applications can

be audited and maintained. Continuous learning is equally important, because tools, platforms and expectations change quickly, and professionals who keep strengthening transferable skills are better able to adapt without depending on any single product or role.

3.9 Likely Future Workflow

A plausible future process for building applications combines AI speed with human accountability. In this model, AI produces a first draft quickly, but people remain responsible for checking and approving it before it reaches users. Figure 3.9 shows one possible cycle; it is an illustrative model rather than a description of any specific organisation's practice.

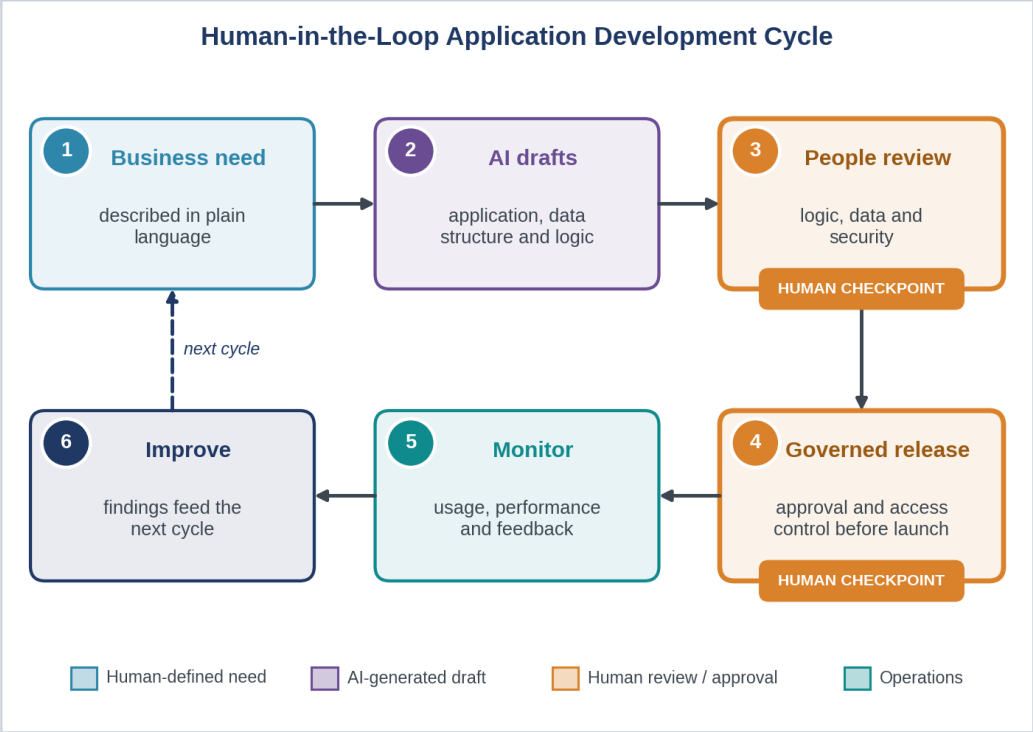


Figure 3.9: Human-in-the-loop cycle for AI-assisted application development

The cycle begins when a business need is described in plain language. AI then drafts the application, including its data structure and logic. People review the logic, data handling and security in the first human checkpoint, and the application is released only after governance and approval, which forms the second checkpoint and includes access control. After release, the application is monitored and feedback is collected, and the resulting improvements feed the next development cycle. The two highlighted checkpoints show that human review and approval remain essential: AI accelerates drafting, but responsibility for correctness, security and compliance stays with people.

3.10 Presentation Outline

The findings of this study can be presented in class as a ten-slide sequence with one main idea per slide. Table 3.10 lists the slide titles and concise key points, so that the presentation moves from definition and background, through analysis, to the practical project and conclusion.

Slide No.	Slide Title	Key Points
1	Title and Introduction	Topic, presenter name and register number; purpose of the study.
2	What Is Low-Code?	Visual building with the option to add code; link to the loan application work.
3	Evolution of Application Development	Timeline from hand coding to AI-assisted and agent-based building.
4	Platform Landscape	Zoho Creator, Power Apps, OutSystems, Mendix, Appian, Salesforce, AppSheet and their different focuses.
5	Future Industry Trends	AI-assisted building, agents, governed citizen development, hyperautomation, composable design.
6	Career Pathways	Entry, intermediate and senior roles; possible switches to other careers.
7	Skills and Learning Roadmap	Six-layer skill stack and five-stage learning plan.
8	Risks and Challenges	Shadow IT, security, AI output quality, lock-in, changing job market and responses.
9	My Practical Work — Bank Loan Application	Bank Loan Application Form from the course experiments: form, workflow and pro-code extension as a hands-on example.
10	Conclusion and Questions	Low-code and pro-code will work together; keep learning; open for questions.

Table 3.10: Ten-slide presentation outline

Each slide should carry only a few short phrases, with the diagrams from Sections 3.1, 3.4, 3.6, 3.7 and 3.9 reused as visuals. Slide 9 links the theory to the course experiments, showing how the practical Bank Loan Application work illustrates the skills discussed in the earlier slides.

4. Final Result

The study showed that low-code development has grown from a quick way of building simple visual applications into a wider field in which visual building, pro-code extension, automation and AI-assisted development work side by side. Early rapid-development tools and visual platforms made application building faster and more accessible, and current developments such as citizen development, hyperautomation and agent-based building extend that direction while increasing the need for review and governance.

The platform landscape is broad. Zoho Creator, Power Apps, OutSystems, Mendix, Appian, Salesforce Platform and Google AppSheet serve different application areas, yet they share data modelling, workflow design, access control and integration concepts. Skills learned on one platform therefore transfer to others, which reduces the risk of depending on a single product.

Career progression runs from citizen developer and junior low-code developer roles, through low-code development, integration and automation, to architecture, platform leadership and AI-automation design, with routes into pro-code engineering, product ownership, consulting and entrepreneurship. Progress along these routes depends on technical foundations, APIs, scripting, AI services, security and governance, and the layered skill stack and five-stage roadmap show how these are acquired in order. The Bank Loan Application Form is a practical example of Stages 2 and 3 of that roadmap and gives concrete material for the final presentation.

24140117